EJS Templates in Express.js (Part 2)

Form Submission, Partials, and Static Files

1. Form submission using EJS
2. Handling form data with Express
3. Using EJS Partials (includes)
4. Serving static files (CSS, images, etc.)

---

1. Creating a Form Page Route

Step 1: Create a Route to Display the Form

In `index.js`:

```
app.get("/form", (req, res) => {
  res.render("form", { message: null });
});
```

- This route renders the form page.
- A `message` variable is passed with `null` initially.

---

2. Designing the Form Using EJS and Bootstrap

Step 2: Add Bootstrap via CDN

Bootstrap is included using a CDN link (no download required):

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap/dist/css/bootstrap.min.css"
rel="stylesheet">
```

Step 3: Create the Form Structure

Key elements:

- Bootstrap container and rows
- Header section
- A form with:
    - Text input for **Name**
    - Submit button

```
<form method="POST" action="/submit">
  <label>Enter Name</label>
  <input type="text" name="myname" class="form-control">
  <br>
  <input type="submit" value="Submit" class="btn btn-primary">
</form>
```

Important Notes:

- `method="POST"` is used for secure data submission.
- `GET` is avoided because it exposes data in the URL.
- `action="/submit"` specifies the route where the form will be submitted.

---

3. Handling Form Submission (POST Route)

Step 4: Create a POST Route

```
app.post("/submit", (req, res) => {
  const name = req.body.myname;
  const message = `Hello ${name}, you submitted the form`;

  res.render("form", { message });
});
```

---

4. Enabling Form Data Parsing (Middleware)

Step 5: Use `express.urlencoded` Middleware

Without this middleware, `req.body` will be `undefined`.

```
app.use(express.urlencoded({ extended: false }));
```

Why this is required:

- It allows Express to read form data from POST requests.
- Necessary for handling both form data and JSON payloads.

---

5. Displaying Submitted Data on the Same Page

Step 6: Show Message Conditionally in EJS

In `form.ejs`:

```
<% if (message) { %>
  <div class="alert alert-success mt-3">
    <%= message %>
  </div>
<% } %>
```

Result:

- Message appears **only after form submission**
- Refreshing the page removes the message

---

6. Understanding EJS Partials (Includes)

What are Partials?

Partials are reusable template parts such as:

- Header
- Footer
- Navigation bar

They prevent code repetition and improve maintainability.

Step 7: Create Partial Files

Inside `views/`:

- `header.ejs`
- `footer.ejs`

Move common HTML (header, Bootstrap links, footer) into these files.

Step 8: Include Partials in Pages

```
<%- include("header") %>
```

```
<%- include("footer") %>
```

Benefits:

- One-time changes apply to all pages
- Cleaner and smaller EJS files

- Improved productivity

---

7. Serving Static Files in Express.js

What Are Static Files?

- CSS
- JavaScript
- Images
- Fonts

Step 9: Create a Public Folder

```
public/
 ├── css/
 │    └── style.css
 ├── images/
 │    └── flower.jpg
```

Step 10: Configure Static Middleware

In `index.js`:

```
app.use(express.static("public"));
```

This tells Express:

> All static assets are located inside the `public` folder.

---

8. Linking CSS Files

Example: `style.css`

```
body {
  background-color: pink;
}
```

Link in EJS:

```
<link rel="stylesheet" href="/css/style.css">
```

✔ No need to mention `public` in the path ✔ Express automatically maps it

---

9. Displaying Images

Image inside `public/images/flower.jpg`

```
<img src="/images/flower.jpg" class="img-fluid">
```

How it works:

- Express looks inside `public/`
- Finds `images/flower.jpg`
- Displays the image successfully

---

10. Using Subfolders for Static Assets

You can organize assets like this:

```
public/
  ├── css/
  ├── js/
  ├── images/
  ├── fonts/
```

No additional configuration is required as long as everything is inside `public`.

---

11. Key Takeaways

Form Submission

- Use `POST` method
- Define `action` route
- Enable `express.urlencoded()`
- Access form data using `req.body`

EJS Partials

- Prevent code duplication
- Use `<%- include("file") %>`
- Ideal for headers, footers, layouts

Static Files

- Use `express.static("public")`

- Link CSS, JS, images without mentioning `public`
- Supports subfolders automatically
